

TPEC

Regular Programs

1. Insertion Sort - 1

```
int j = ar_size-1;
int v = ar[j];
while(v < ar[j-1]) {
    ar[j] = ar[j-1];
    j--;
    print(ar_size, ar);
}
ar[j] = v;
print(ar_size, ar);
```

2. Insertion Sort - 2

```
for (int i = 1; i < ar_size; ++i) {
    int j = i - 1;
    int p = ar[i];
    while (j >= 0 && p < ar[j]) {
        ar[j+1] = ar[j];
        j--;
    }
    ar[j+1] = p;
    printf("%d", ar[0]);
    for (int k = 1; k < ar_size; ++k) {
        printf(" %d", ar[k]);
    }
    printf("\n");
}
```

3. Compare the Triplets

```
int a_score = 0;
int b_score = 0;
scanf("%d %d %d", &b0, &b1, &b2);
if (a0 > b0)
    a_score++;
else if (a0 < b0)
```

```

        b_score++;
    else{}
    if (a1 > b1)
        a_score++;
    else if (a1 < b1)
        b_score++;
    else {}
    if (a2 > b2)
        a_score++;
    else if (a2 < b2)
        b_score++;
    else {}

```

4. Diagonal Difference

```

for(firstarray = 0; firstarray < n; firstarray++) {
    for(secondarray = 0; secondarray < n; secondarray++) {
        scanf("%d",&a[firstarray][secondarray]);
    }
}
while(i<n) {
    RightDiagonalSum=RightDiagonalSum+a[i][i];
    i++;
}
j=n-1,i=0;
while(i<n) {
    LeftDiagonalSum=LeftDiagonalSum+a[i][j];
    i++;
    j--;
}

```

5. Counting Sort - 1

```

scanf("%d",&n);
for(i=0;i<100;i++) {
    b[i]=0;
}
for(i=0;i<n;i++) {
    scanf("%d",&a);
    b[a]++;
}
for(i=0;i<100;i++) {

```

```

    printf("%d ", b[i]);
}

```

6. Non-Divisible Subset

```

scanf("%d %d", &n, &k);
int mods[k];
for (int i = 0; i < k; i++) {
    mods[i] = 0;
}
for (int i = 0; i < n; i++) {
    scanf("%d", &a);
    mods[a % k]++;
}
total += min(1, mods[0]);
for (int d = 1; d < (k + 1) / 2; d++) {
    total += max(mods[d], mods[k - d]);
}
if (k % 2 == 0) {
    total += min(1, mods[k / 2]);
}

```

7. Counter Game

```

scanf("%d", &t);
for(i=0; i<t; ++i) {
    win=0;
    scanf("%lu", &n);
    if(n==1)
        printf("Richard\n");
    else {
        while(n!=1) {
            if(isPow2(n))
                n>>=1;
            else
                n-=largePow(n);
            ++win;
        }
    }
    if(win%2==0)
        printf("Richard\n");
    else
        printf("Louise\n");
}

```

```

}

return 0;

...

...

return !(n&(n-1));

```

8. Bear and Steady Gene

```

int target = n / 4;
int count[128] = {0};
for (int i = 0; i < n; i++) {
    count[gene[i]]++;
}
if (is_balanced(count, target)) {
    return 0;
}

...

...
for (int right = 0; right < n; right++) {
    count[gene[right]]--;
    while (is_balanced(count, target)) {
        if ((right - left + 1) < min_len) {
            min_len = (right - left + 1);
        }
        count[gene[left]]++;
        left++;
    }
}
return min_len;
}

```

9. Caesar Cipher

```

x=ar[i];
if(x>=97 && x<=122) {
    x=x+(k%26);
    if(x>122) {
        x=96+(x-122);
    }
    ar[i]=x;
}
else if(x>=65 && x<=90) {

```

```

x=x+(k%26);
if(x>90) {
    x=64+(x-90);
}
ar[i]=x;
}

```

.0. Pangrams

```

if(s[i]==' ')
    continue;
if(s[i]>='a')
    s[i]=s[i]-('a'-'A');
if(!(f[s[i]]++))
    ans++;

```

.1. Prefix Sum Array

```

int i;
for (i = 1; i<n; i++) {
    arr[i] = arr[i] + arr[i-1];
}
...
...
...
int result;
if (a == 0)
    result = arr[b];
else
    result = arr[b] - arr[a-1];
printf("The sum is %d", result);

```

.2. Fenwick Tree Construction

```

int i, sum=0;
for (i = a; i<=b; i++) {
    sum += arr[i] ;
}
return sum;
...
...
...

```

```

for (k=1; k<=n; k++) {
    a = k - (k&-k) + 1;
    b = k;
    T[k] = sum(arr,n,a,b);
}

```

.3. Fenwick Tree - Sum of Elements

```

int s = 0;
while (k >= 1) {
    s += T[k];
    k -= k&-k;
}
printf("\n The sum is %d",s) ;

```

.4. Sherlock and Cost

```

X = L - 1 + ML;
Y = R - 1 + MR;
P = abs(L - B) + ML;
Q = abs(R - B) + MR;
ML = (X > Y ? X : Y);
MR = (P > Q ? P : Q);
...
...
...
ML = MR = 0;

```

.5. Marc's Cakewalk

```

quicksort(x, start, i-1);
quicksort(x, i+1, end);
...
...
...
quicksort(calories, 0, n-1);
for(i = 0; i < n; i++){
    sum += calories[i] * (1 << i);
}
printf("%d\n", sum);

```

Additional Programs

1. Closest Numbers

```
quickSort(a,0,n-1);
int min=a[1]-a[0];
for(i=2;i<n;i++)
    if(a[i]-a[i-1]<min) min=a[i]-a[i-1];
for(i=1;i<n;i++)
    if(a[i]-a[i-1]==min) printf("%d %d ",a[i-1],a[i]);
printf("\n");
```

2. Ice Cream Parlour

```
scanf("%d%d",&c,&l);
for(i=0;i<l;i++)
    scanf("%d",&arr[i]);
for(i=0;i<l-1;i++)
    for(j=i+1;j<l;j++)
        if(arr[i] + arr[j] == c)
            printf("%d %d\n", i+1, j+1);
```

3. Maximizing XOR

```
int max = 0,i,j;
for(i=l;i<r;i++)
    for(j=i+1;j<=r;j++)
        max = max<(i^j)?i^j:max;
return max;
```

4. Sum vs XOR

```
while(n>0){
    if(n%2==0)m*=2;
    n/=2;
}
printf("%lld\n",m);
```

5. Flipping Bits

```
scanf("%d", &t);
while(t-- > 0) {
    scanf("%u", &n);
```

```

    printf("%u\n", ~n);
}

```

6. Mark and Toys

```

int main()
{
    int n, k, i, avail = 0, count = 0;
    int cost[100000];
    scanf("%d%d", &n,&k);
    for(i = 0; i < n; i++)
        scanf("%d", &cost[i]);
    quicksort(cost, 0, n - 1);
    while(count < n && avail + cost[count] <= k)
    {
        avail += cost[count];
        count++;
    }
    printf("%d\n", count);
    return 0;
}

```

7. Greedy Florist

```

int a[200],i,j,k,l,sum,n;
...
...
...
sum = 0;
for(i=0;i<n;i++)
    sum+= a[i]*(1+i/k);
printf("%d\n",sum);

```

8. CamelCase

```

scanf("%s",s);
int count=0,i;
for(i=0;i<strlen(s);i++){
    if(s[i]>=65 && s[i]<=90){
        count++;
    }
}

```

```

    }
    printf("%d\n", count+1);

```

9. Mars Exploration

```

for(i=0; S[i]!='\0'; i+=3){
    if(S[i]!='S'){
        count++;
    }
    if(S[i+1]!='O'){
        count++;
    }
    if(S[i+2]!='S'){
        count++;
    }
}

```

10. Funny String

```

funny = 1;
for(i=1, r=N-2; i<N; i++, r--) {
    if (fabs(S[i]-S[i-1]) != fabs(S[r]-S[r+1])) {
        funny = 0;
        break;
    }
}
if (funny){
    printf("funny");
}
else
    printf("not funny");

```

11. Alternating Characters

```

while(t--){
    scanf("%s", c);
    for(i=0; c[i]!='\0'; i++){
        if(c[i]==c[i+1]){
            count++;
        }
    }
}

```

.2. Beautiful Binary String

```
if(B[i]=='0' && B[i+1]=='1' && B[i+2]=='0'){  
    B[i+2]='1';  
    count++;  
}  
i++;
```

.3. Fenwick Tree - Add a value

```
while (k <= n) {  
    T[k] += x;  
    k += k&-k;  
}  
display(T,n);  
...  
...  
...  
printf("\n Enter the position and value to be increased ");  
scanf("%d%d",&k,&x);  
add(T,n, k, x);
```